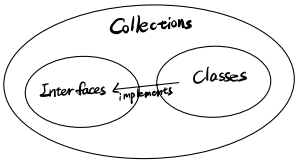
L0: ① Data structures } Program
② Algorithm }

L1: ① Abstraction → identify which specific information is important for the user of a module, and which information is unimportant → application { Data mining / Big Data Analytics / Knowledge Discovery in Databases (KDD)
② Information hiding → all unimportant information should be hidden from a user (principle)
③ Encapsulation → hide what should be hidden, make visible what is intended to be visible (technique)
④ Efficiency
⑤ tatic / Dynamic

L2: Collections { Interfaces: 接口是一种描述类应该具有哪些方法的抽象类型
↑ implements
Classes: 类实现接口定义的方法
→ Interfaces 接收 Parameter Types (参数化类型 / 泛型类型)
没有 constructors, fields, method bodies
Classes 定义

Collections: 用来储存和操作多个元素的数据结构
是用于存储和操作数据的一组接口和类的集合



Interfaces
L3: ① Collection<E>
↑ extends
List<E>
implements
ArrayList<E>
LinkedList<E>
Classes

type of the value
private List<Task> tasks;
type of collection
tasks = new ArrayList<Task>();
type of the value
class

② Iterator ← Interface
提供了用于遍历集合 (e.g. 列表、集合等) 中元素的方法

L4: ① Bags
no standard implements
② Set
③ Stack
Stack implements List (Stack class 继承自 Vector, Vector implements List)
Applications { Processing files of structured (nested) data
Program execution
Undo in editors
Expression evaluation
↓
Infix expression → Postfix expressions
中缀 后缀
对于后缀无括号 → 第一步研究括号是否 Balanced
Balanced 才可转化: 根据运算顺序增加括号, 再将运算符放至该层括号后, 最后去掉括号
e.g. a-b+c → ((a-b)+c) → ((ab)-c)+ → ab-c+
(a+n)×(b-&×m) → ((a+n)×(b-(&×m))) → ((an)+(b(&m)x)-)x → an+b&mx-x
为什么? 可见 Evaluating Postfix Expression
④ Maps

L5: ① Maps
② Queues
a Queue interface
classes: LinkedList, PriorityQueue
③ Iterators and Iterable 都是接口
提供了遍历集合元素的方法
代表了一个包含一组元素的集合, 它允许对其进行迭代操作
实现其的类可以使用增强 for 循环
有一个 iterator() 并返回了一个实现 Iterator 的对象

For { Collection objects
Anything that generates a sequence of values
Scanner
Pseudo Random Number generator

L6: ① Iterators and Iterable
② Sorting Collections
Comparable<T> is an Interface
定义在要比较的类的内部, 表示该类有比较能力, 可以使用 object 的 compareTo 方法去排序
解决 natural order
若 specified order
↓
Compactor 接口 → 一个独立的比较器, 可以实现多个不同的排序规则
用于不改变对象本身的情况下, 定义不同的比较规则
多个不同的比较器对同一类的对象进行不同的排序
↓
compareTo(Object): int
compare(Object, Object): int

L7: ① Comparators
② Exceptions
{

throw
catch

③ Interface ← implements — class

List<E> extends Collection<E>
ArrayList<E> implements List<E>

L8: ArrayList

List中有太多方法需定义，且许多是复杂的

↓

an Abstract class { Defines the complex methods in term of the basic methods
Leave the basic methods "abstract" (no body)
classes implementing List can extends the abstract class.

↓

AbstractList<E> implements List<E>

No constructor or fields
declared abstract → 该方法是一个抽象方法，需要在子类中进行重新定义

子能实例化

↓ 5有方法的 Signature

ArrayList<E> extends AbstractList<E>

L9: ArrayList → iterator( )

L10: ① Cost → measure number of steps

↓

using asymptotic notation → Big Oh Notation

↓

step

② ArrayList: { get and set O(1)
remove O(n)
add (at i) O(n)
add (at ends) O(1) (average) O(n) (worst) O(1) (amortised average)

③ ArraySet O(n) (add/ find/ remove)

↓

cost of searching

L11: Recursion

examples { factorial
fibonacci

can be rewritten as an iterative function

{ iteration: efficient but not easy to design
recursion: slow but elegant

L12: ① Testing { Black box
White box

② Queue

Arrays stored in contiguous chunks of memory

↓

insert slow

↓

Linked Structure

③ Linked Node

L13+14: To represent the empty list as an object

↓

separate "header" object to represent a list

↓

LinkedList<E> extends AbstractList<E>

↳ has a inner class: Node

Cost

L15: ①用 LinkedList 的尾部作为 Stack 的顶导致 Push、Pop 很慢

↓

用 LinkedList 的头作为 Stack 的顶

↓

LinkedStack<E> extends AbstractCollection<E>

②无论将 Queue 的头放在 list 的头还是尾都会导致某一个操作复杂

↓

Have pointers to both ends

↓

LinkedQueue<E> implements AbstractQueue<E>

L16: ArraySet contains, add, remove: O(n)

↓

slow due to searching

↓

so speed up searching

↓

binary search

↓

Sorted ArraySet (with Binary Search) { contains O(log(n))
add O(n) > 因为涉及移动
remove O(n)

若需多次 add( ) / remove( ), ArraySet 更好，因为无需排序

Linear Collections

1- dimensional data structure

避免construct时一个个添加，新的Constructor on P21

L17: 一个个 slow $O(n^2)$，一起 $O(n\log(n))$ → P9

ways of sorting { Selection Sort $O(n^2)$ / Bubble Sort $O(n^2)$ / Insertion Sort $O(n^2)$ } slow

L18 Quick Sorting { Merge Sort $O(n \log n)$ / Quick Sort $O(n\log n)$ (average) $O(n^2)$ (worst) }

L19 ArraySet $O(n)$
SortedArraySet Search: $O(\log(n))$ (binary search cut of inserting)
Add/Remove: $O(n)$
Add n items: $O(n^2)$
Initialise with n items: $O(n\log(n))$ fast sorting

L20 Trees
general trees

L21 ① Linked List $O(N)$ for insert, lookup and remove
↓
Binary Search Tree (BST) $O(\log N)$ in average for insert, lookup and remove
$O(n)$ worst case → degenerates to a linked list

② Traversal { in-order: left, root, right / pre-order: root, left, right / post-order: left, right, root / Breadth-First }

③ Balanced Search Trees (rotation makes banlance to avoid)
AVL Trees (Adelson-Velskii and Landis) { search $O(\log n)$ / insert $O(\log n)$ / delete $O(\log n)$ / space $O(n)$ }

L22: ① BinaryTree
② GeneralTree

L23: ① Hash Table 键通过Hash Function处理获得索引，将值存到对应索引的位置 → { collision / size }

② Hash Function → { avoid collisions / spread keys evenly in the array / inexpensive to compute $O(1)$ }
for { Integer / String }

③ Open addressing { linear probing → primary clustering / quadratic probing → secondary clustering / double probing → avoid clustering }

④ Separate chaining

L24: Graph
subgraph
spanning subgraph

L25: ① Trails, Paths, Circuits
② Connected Graph
③ Incidence matrix, Adjacency matrix

L26: ① Trees
↓
② Spanning Trees
↓
③ Minimum Spanning Tree

L27: Shortest Path

Hierarchical collections